MedSync: Medication Reminders and Care Teams

Project documentation for the Afro-Asian Forum for Innovation and Technology (AFFIAT), Aswan Finals 2026. By Sief & Leen.



TECHNOFuture
KAFR EL SHEIKH



MedSync

> **About this document:** the technical facts were taken from the code, so check them if you change the app. After any change, run `python docs/build.py` to rebuild the PDF, the poster and the slides.

1. Project Overview | نظرة عامة على المشروع

Field	Value
Project Title	MedSync: Medication Reminders and Care Teams
Project Type	Web application
Description	MedSync is a web app that helps patients take their medicines on time. Doctors prescribe medicines and set dose times, patients get an alarm at each dose, and nurses record every dose as given or missed.
Target Users	Patients who take medicines every day, and the doctors and nurses who care for them
Event	AFFIAT · Afro-Asian Forum for Innovation and Technology · Aswan Finals 2026
Live app	Not deployed yet. The demo runs locally with `python -m streamlit run app.py`.

Tech Stack | التقنيات المستخدمة

Category	Technology	Purpose
Language	Python	All the app's logic

Category	Technology	Purpose
Framework	Streamlit (tested with 1.66)	Pages, forms, tabs, the chat box and session state
Database	SQLite (built into Python)	Accounts, care teams, prescriptions, doses and visit notes in `database.db`
AI	Google Gemini through `google-genai`	Answers chatbot questions that the database can't
Styling	CSS inside `st.markdown` + `.streamlit/config.toml`	The dark theme, teal headings and buttons, and the medicine cards (`style.py`)
Alarm	HTML + JavaScript in `components.html`	Checks the clock in the patient's browser, shows a banner, beeps and sends a notification (`alarm.py`)

Dependencies | المكتبات

Package	Version	Purpose
streamlit	Not pinned (tested with 1.66.0)	The whole web interface
google-genai	Not pinned (tested with 2.31.0)	Calls Gemini from the chatbot. The app still runs without a key.
sqlite3	Standard library	Talking to the database. Nothing to install.
json, datetime, os	Standard library	Passing medicine times to the alarm, today's date for doses, file paths

2. Problem Statement | المشكلة

Medicine only works if you take it. Patients who take medicine every day, often for a chronic illness, miss doses or take them late. Nobody records it, so the doctor finds out only at the next visit.

Why does it matter? | لماذا هذا مهم؟

When doses are missed, the treatment stops working and the patient can get worse. Without a record, the doctor can't tell whether the medicine failed or was simply not taken.

How is it currently solved? | كيف تُحل المشكلة حاليًا؟

Today people rely on paper charts in clinics and alarms on their phones at home. A phone alarm tells the doctor nothing, and paper charts are slow to read and easy to lose.

3. Technical Architecture | البنية التقنية

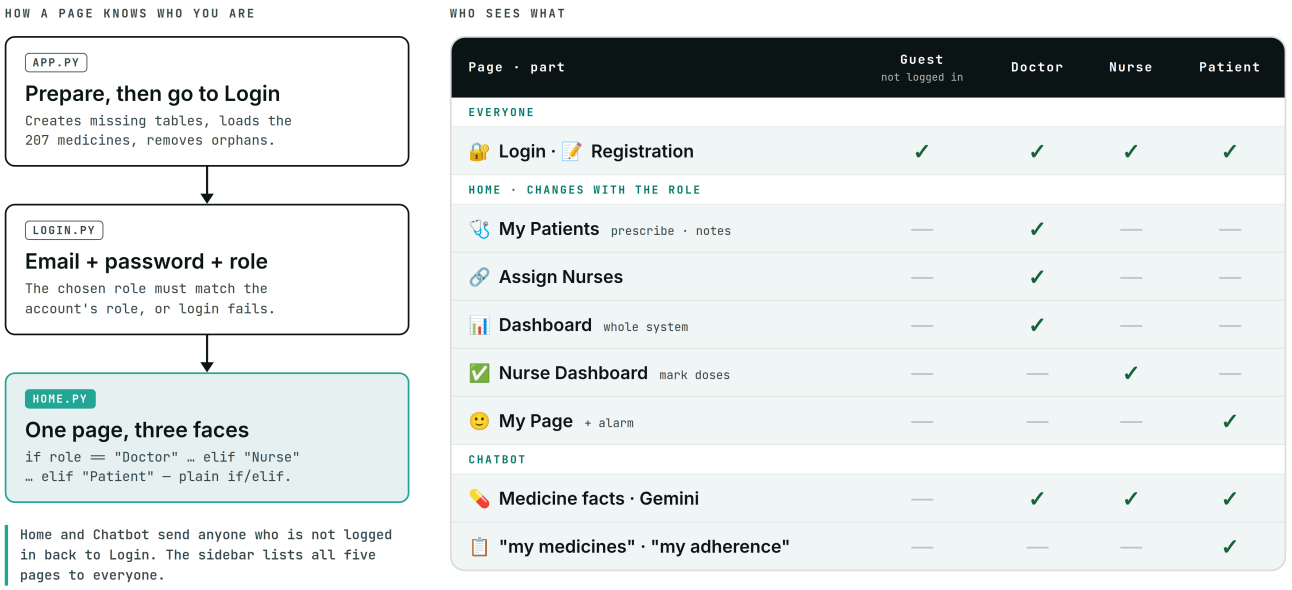
Project Structure | هيكل المشروع

```
medisync/
├── app.py                 # Entry point: prepares the database, then opens Login
├── database.py            # Every SQL statement in the app (9 tables, no JOIN)
├── database.db            # The SQLite database the app uses
├── medicines_seed.sql     # 207 medicines, loaded the first time the table is empty
├── style.py               # Shared dark theme: teal headings, buttons and cards
├── alarm.py               # The dose alarm that runs in the patient's browser
├── beep_data.py           # The beep sound, as base64 WAV text
├── requirements.txt       # streamlit, google-genai
├── .streamlit/
│   ├── config.toml        # Theme colours
│   └── secrets.toml       # GEMINI_API_KEY (you create it from secrets.toml.example)
├── pages/
│   ├── login.py           # Email + password + role
│   ├── registration.py    # New account, with extra fields per role
│   ├── home.py            # Doctor / Nurse / Patient pages
│   └── chatbot.py         # Database answers + Gemini
└── docs/                  # This documentation, the poster, the slides and the diagrams
```

Architecture and Data Flow | تدفق البيانات

```
User's browser
    → app.py: init_db() + seed_medicines_if_empty(), then st.switch_page("pages/login.py")
    → login.py: checks email + password in users, and that the chosen role matches
        → saves logged_in, user_id, role and name in st.session_state
    → home.py: if role == "Doctor" / "Nurse" / "Patient" → that role's page
        → calls functions in database.py
            → each function opens sqlite3, runs one plain SQL statement, commits, closes
            → tables are linked in Python with for loops (no JOIN)
    → patient page also sends the medicine times to alarm.py (JavaScript in the browser)
```

All SQL lives in `database.py`. The pages never write SQL themselves; they only call functions like `add_prescription()` or `get_adherence_for_patient()`.



Key Files | الملفات الأساسية

File	Purpose
`app.py`	Sets the page title and 💊 icon, creates any missing tables, removes orphaned profile rows, loads the medicine list, and opens Login.
`database.py`	The only file with SQL: `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`. Also works out adherence and sorts history newest first with a bubble sort.
`pages/home.py`	The biggest page. One `if/elif` on the role draws the Doctor page (3 tabs), the Nurse page or the Patient page.
`alarm.py`	Builds an HTML + JavaScript widget. Every 15 seconds it compares the clock with each medicine's time and, once a day per medicine, shows a banner, beeps and sends a browser notification.
`pages/chatbot.py`	Answers greetings, `help`, `my medicines`, `my adherence` and medicine names from the database. Anything else goes to Gemini.

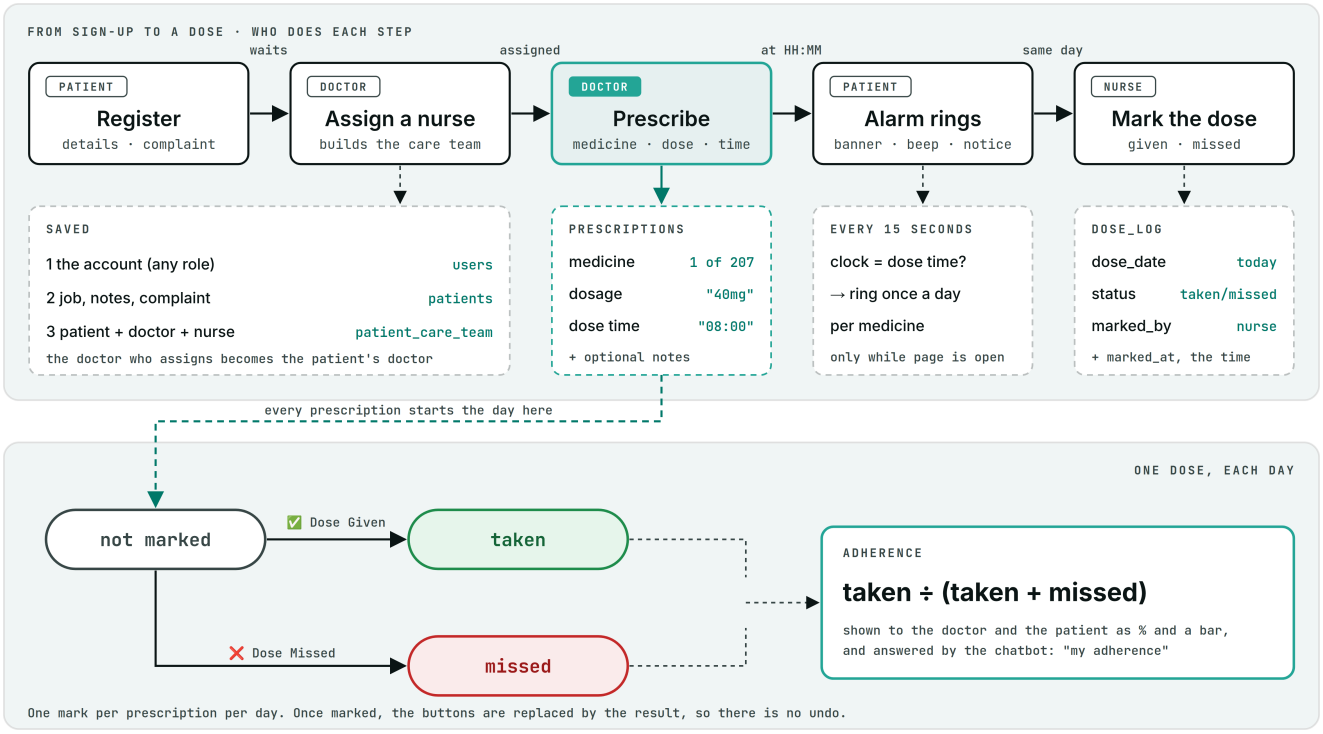
4. Features & User Flow | المميزات ورحلة المستخدم

Feature List | قائمة المميزات

#	Feature	Description	Status
1	Login with role	Email, password and the role you log in as. The role must match the account.	Complete. Passwords are plain text (see Limits).
2	Registration	Basic details for everyone, plus specialization, license and experience for doctors, license for nurses, and job, notes and complaint for patients.	Complete. Any role can be chosen.

#	Feature	Description	Status
3	Care teams	A doctor picks a nurse for each patient who has none. That doctor becomes the patient's doctor.	Complete. No way to change a team in the app.
4	Prescribe	Search 207 medicines by name, then set the dosage, dose time and notes.	Complete. One dose time per prescription.
5	Delete a prescription	A Delete button next to each medicine on the doctor's tab.	Complete
6	Visit notes	A diagnosis and notes for each visit, with the date, newest first.	Complete
7	Adherence	Doses taken ÷ all marked doses, as a %, a bar and "x taken, y missed".	Complete
8	Doctor dashboard	Counts of doctors, nurses and patients, doses given and missed today, and each doctor's and nurse's patient count.	Complete
9	Nurse dose marking	"Dose Given" or "Dose Missed" for each of today's medicines, with a search by patient name.	Complete. A mark can't be undone.
10	Patient page	Care team, medicines and times, adherence, full dose history and visit notes.	Complete
11	Dose alarm	Banner, beep and browser notification at the dose time.	Complete. Only while the page is open.
12	Chatbot	Medicine facts and the patient's own medicines and adherence from the database; other questions go to Gemini.	Complete. Gemini needs a key and internet.

User Flow | رحلة المستخدم




```
Patient: Registration (pages/registration.py) → waits without a care team
Doctor:  Home → Assign Nurses → picks the patient and a nurse → patient_care_team row
Doctor:  Home → My Patients → picks the patient
             → Prescribe: medicine + dosage + time "HH:MM" → prescriptions row
             → Add Visit Note: diagnosis + notes → visit_notes row
Patient: Home → My Page → alarm widget watches each time
             → at HH:MM: banner + beep + "MedSync Reminder: Time to take <medicine>
(<dosage>)"
Nurse:   Home → Nurse Dashboard → Dose Given / Dose Missed → dose_log row
Doctor and patient: adherence % = taken ÷ (taken + missed)
```

Screen Reference | الشاشات

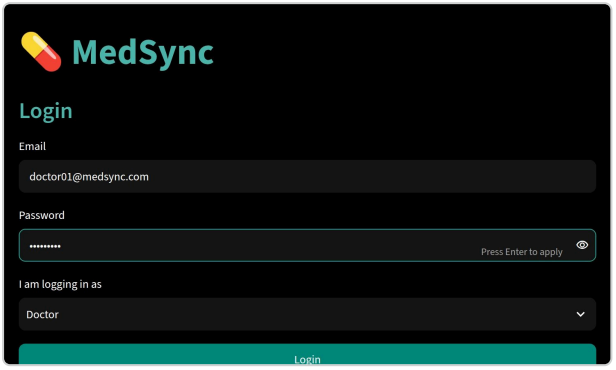
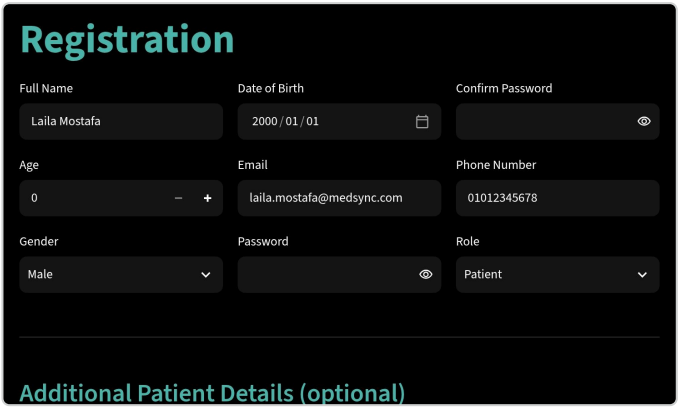
Screen	File	Who sees it	Description
Login	`pages/login.py`	Everyone	Email, password, role dropdown, and a "Create new account" button.
Registration	`pages/registration.py`	Everyone	Three columns of basic details, then extra fields for the chosen role.
My Patients	`pages/home.py`	Doctors	Patient search and dropdown, complaint, adherence, medicines with Delete, prescribe form, visit notes.
Assign Nurses	`pages/home.py`	Doctors	Patients with no care team, a nurse dropdown and "Assign Nurse to this Patient".
Dashboard	`pages/home.py`	Doctors	System counts, today's doses, and a card for each doctor and nurse.

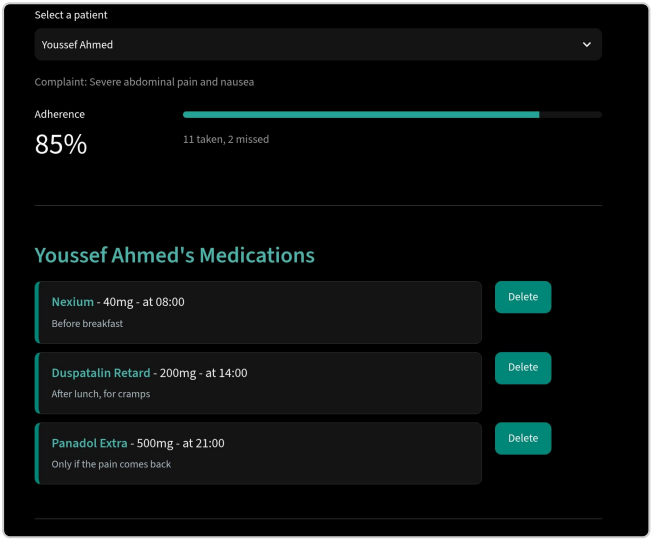
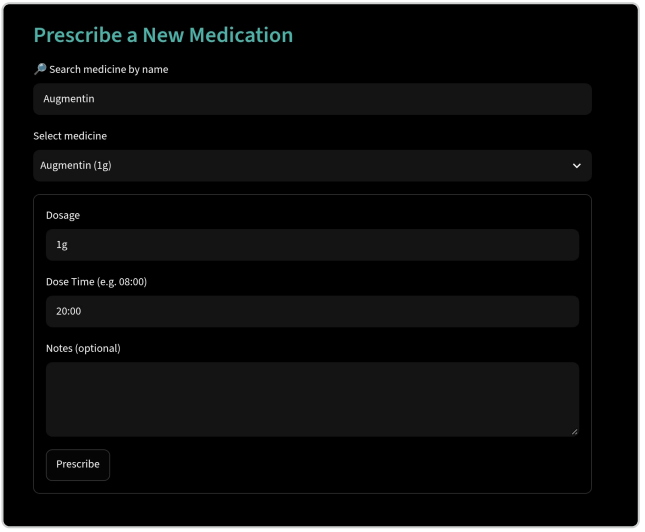
Screen	File	Who sees it	Description
Nurse Dashboard	`pages/home.py`	Nurses	Refresh, patient search, and each patient's medicines with Given / Missed buttons.
My Page	`pages/home.py`	Patients	Care team, medicines (with the alarm), dose history with adherence, visit notes.
Chatbot	`pages/chatbot.py`	Logged in	Chat history and a message box. `my medicines` and `my adherence` answer patients only.

5. UI/UX Design | تصميم واجهة المستخدم

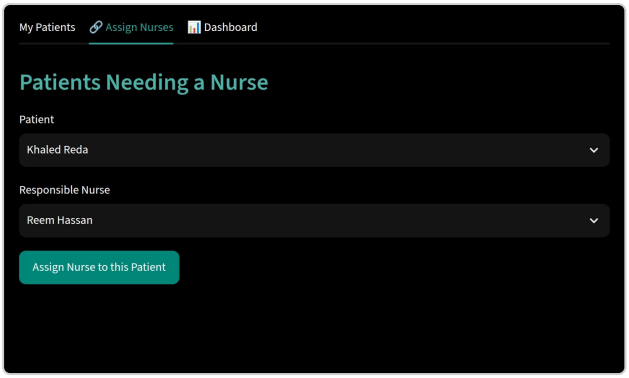
Screenshots | لقطات الشاشة

These come from a copy of the app with demo data for the patient Youssef Ahmed (`patient11@medsync.com`).

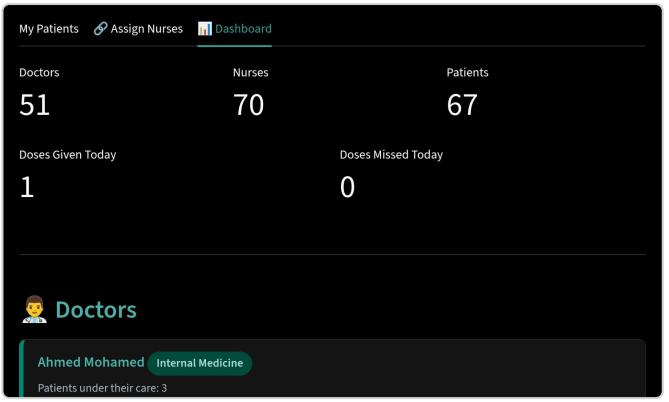
Login	Registration
	

My Patients: adherence and medicines	My Patients: prescribe a medicine
	

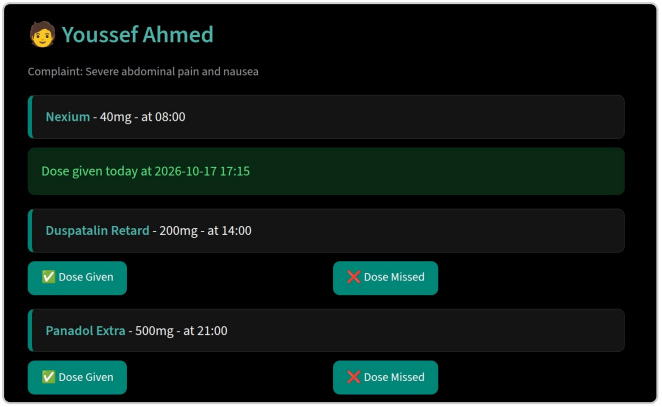
Assign Nurses



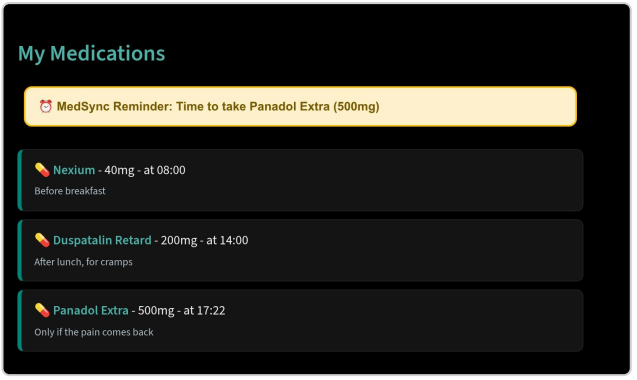
Dashboard

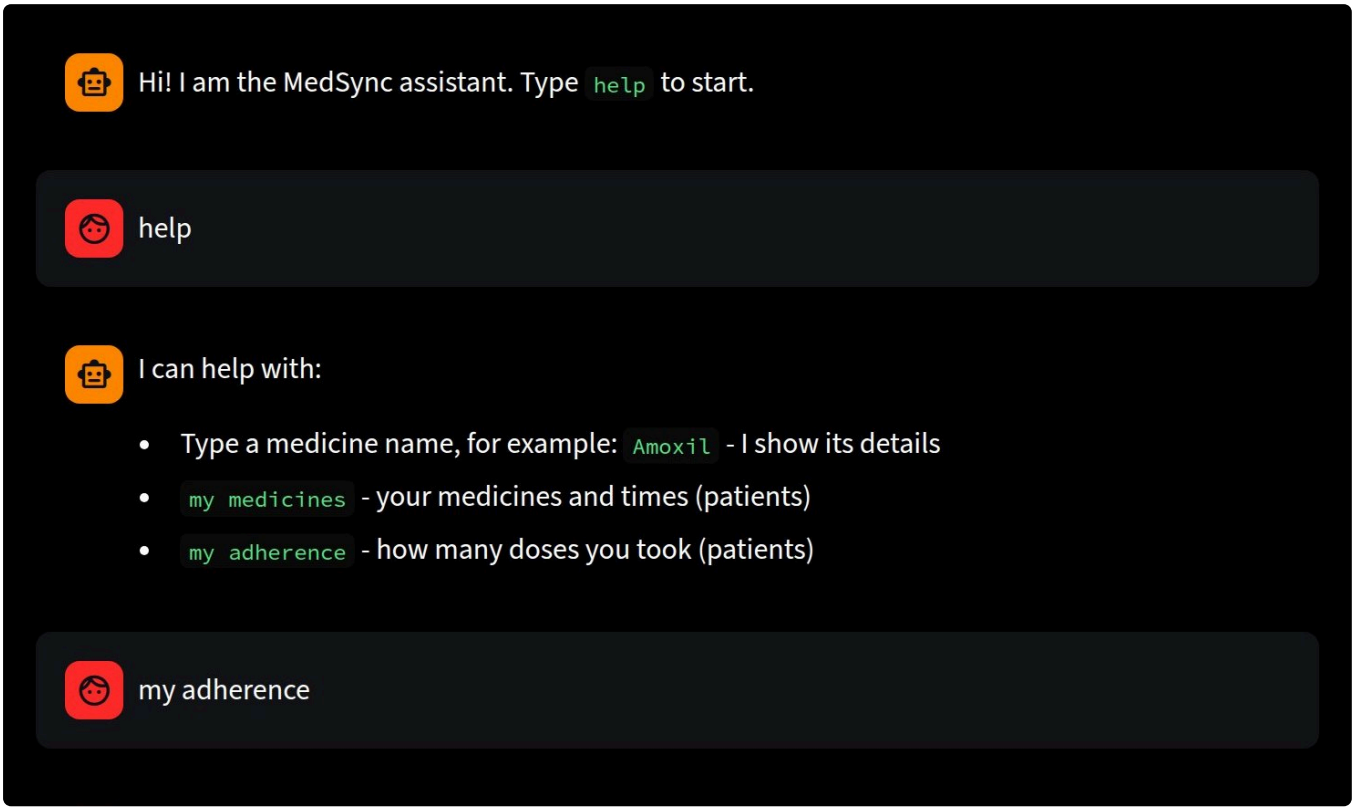


Nurse Dashboard



Patient: alarm banner





Design System | نظام التصميم

Element	Current implementation
Logo	The app uses the 💊 emoji as its page icon and in the "💊 MedSync" title. The capsule mark in this document (`docs/assets/mark.svg`) was drawn for the showcase from that emoji.
Theme	Streamlit dark theme in `.streamlit/config.toml`: background `#000000`, panels `#161616`, text `#FFFFFF`, primary `#26A69A`
Headings	Teal `#4DB6AC` (`style.py`)
Buttons	Teal `#00897B`, darker `#00695C` on hover, 8px rounded corners
Cards	`.med-card`: `#161616` with a 5px `#00897B` left border, used for medicines, visit notes, doctors and nurses
Badges	`.role-badge`: `#004D40` pill with `#B2DFDB` text, used for a doctor's specialization and a nurse's license
Alarm banner	Yellow `#FFF3CD` with a `#FFC107` border, so it stands out from the dark page
Status colours	Streamlit's green `st.success` for a taken dose, red `st.error` for a missed one
Font	Streamlit's default font in the app. The poster and slides use Sora, Inter and JetBrains Mono.
Layout	Centered pages, a sidebar with the user's name, role and Log out, and tabs on the Doctor page

6. Data Sources | مصادر البيانات

Data Sources | مصادر البيانات

Source	Type	Description
Registration form	User input	Name, age, gender, date of birth, phone, email, password and role go into `users`; role details go into `doctors`, `nurses` or `patients`.
Doctor's forms	User input	Care teams, prescriptions and visit notes.
Nurse's buttons	User input	One `dose_log` row for each dose marked.
`medicines_seed.sql`	Seed data	207 medicines from 27 companies, with dosage, frequency, side effects and expiry date. Loaded only when `medicines` is empty.
`database.db`	Demo data	189 accounts (42 doctors, 70 nurses, 77 patients), 76 care teams, and one prescription, dose and visit note. Logins follow the pattern `doctor01@medsync.com` / `Doctor@01`, `nurse01@medsync.com` / `Nurse@01`, `patient01@medsync.com` / `Patient@01`.

Database Schema | قاعدة البيانات

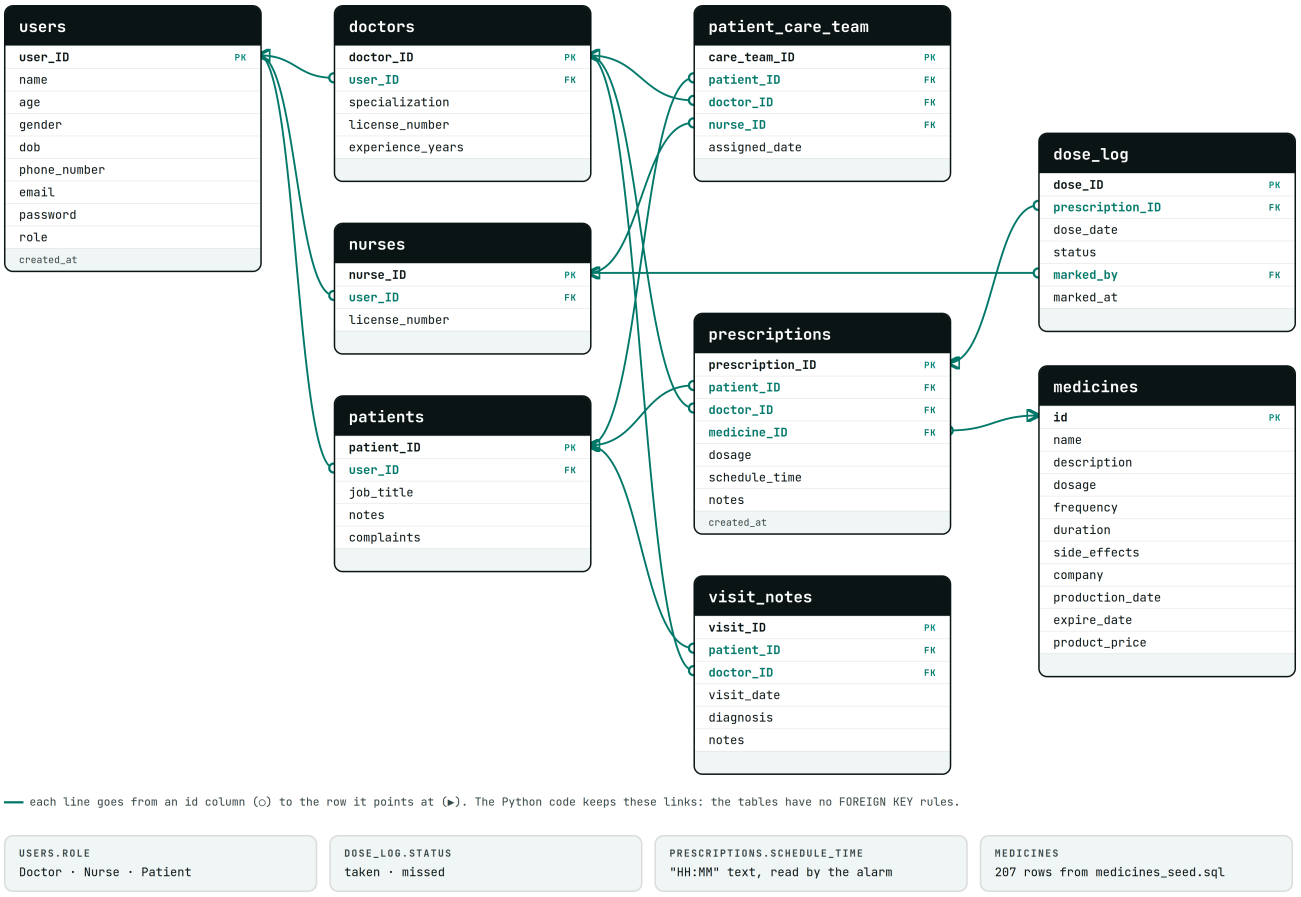


Table	Columns	Description
`users`	`user_ID`, `name`, `age`, `gender`, `dob`, `phone_number`, `email`, `password`, `role`, `created_at`	Every account. `role` is Doctor, Nurse or Patient.
`doctors`	`doctor_ID`, `user_ID`, `specialization`, `license_number`, `experience_years`	A doctor's extra details.
`nurses`	`nurse_ID`, `user_ID`, `license_number`	A nurse's extra details.
`patients`	`patient_ID`, `user_ID`, `job_title`, `notes`, `complaints`	A patient's extra details.
`patient_care_team`	`care_team_ID`, `patient_ID`, `doctor_ID`, `nurse_ID`, `assigned_date`	Who looks after each patient.
`medicines`	`id`, `name`, `description`, `dosage`, `frequency`, `duration`, `side_effects`, `company`, `production_date`, `expire_date`, `product_price`	The ready-made medicine list.
`prescriptions`	`prescription_ID`, `patient_ID`, `doctor_ID`, `medicine_ID`, `dosage`, `schedule_time`, `notes`, `created_at`	A medicine for a patient at one time of day.

Table	Columns	Description
`dose_log`	`dose_ID`, `prescription_ID`, `dose_date`, `status`, `marked_by`, `marked_at`	One row per marked dose. `status` is taken or missed; `marked_by` is the nurse.
`visit_notes`	`visit_ID`, `patient_ID`, `doctor_ID`, `visit_date`, `diagnosis`, `notes`	A doctor's notes for each visit.

The tables have no `FOREIGN KEY` rules. The links in the diagram are kept by the Python code, and `repair_orphan_records()` removes doctor, nurse and patient rows whose account no longer exists.

External APIs | الخدمات الخارجية

API	Purpose	Rate Limit
Google Gemini (`google-genai`, model `gemini-3.5-flash`)	Answers chatbot questions the database can't. The key is read from `GEMINI_API_KEY` in `.streamlit/secrets.toml`.	Set by the Google AI Studio plan of the key; the app does not limit it.
Browser Notification API	The dose reminder pop-up. It runs inside the browser; nothing is sent to a server.	Not applicable

Limits | حدود المشروع

These shortcuts keep the code simple. They are fine for a school demo, not for real patients.

- Passwords are saved as plain text.
- Anyone can register as a doctor or a nurse; the license number is not checked.
- Every doctor sees the whole-system dashboard and can take any unassigned patient.
- The dashboard counts profile rows: it shows 51 doctors and 70 nurses, but 10 doctor rows and 10 nurse rows in the demo data belong to accounts with another role.
- The alarm only works while the patient's page is open, and the dose time must be typed exactly as `HH:MM` (for example `08:00`, not `8:00`).
- Each prescription has one dose time. A medicine taken twice a day needs two prescriptions.
- A nurse's mark can't be changed, and deleting a prescription also removes its doses from adherence.
- Names, notes and complaints are put into the HTML cards without escaping.
- Reloading the page logs you out, because the login lives in `st.session_state`.

7. Educational Content | المحتوى التعليمي

Learning Objectives | أهداف التعلم

Through building this project, a student can practice:

1. Building a multi-page web app with Streamlit, using forms, tabs and session state.
2. Designing SQLite tables and linking them with for loops instead of JOIN.
3. Checking a login and showing each role (doctor, nurse, patient) its own page.
4. Running JavaScript inside a Streamlit page to make a timed alarm with sound and notifications.
5. Calling an AI service (Gemini), with an offline fallback when there is no key or no internet.

8. Marketing Plan | خطة تسويقية

Target Audience | الجمهور المستهدف

- **Primary:** patients who take medicines every day, especially for chronic illnesses.
- **Secondary:** clinics and hospital wards, with their doctors and nurses.

Platforms | المنصات

- Web browsers on hospital computers and phones. MedSync is a web app, so there is nothing to install.
- Shown at the Afro-Asian Forum for Innovation and Technology (AFFIAT), Aswan Finals 2026, then at our school.

9. Student Worksheet | ورقة عمل الطالب

Our Project Information | معلومات مشروعنا

Student Names: Sief & Leen
Project Title: MedSync: Medication Reminders and Care Teams

What is your project about? | عن ماذا يتحدث مشروعك؟

MedSync reminds patients to take their medicines on time and lets nurses record every dose, so the doctor can see who is on track.

What problem does it solve? | ما المشكلة التي يحلها؟

Missed doses. Patients forget their medicines, and nobody knows until the next visit.

Our Data | بياناتنا

Where does data come from?

Doctors, nurses and patients type it in through the forms. The project came with 207 medicines in `medicines_seed.sql` and demo accounts in `database.db`.

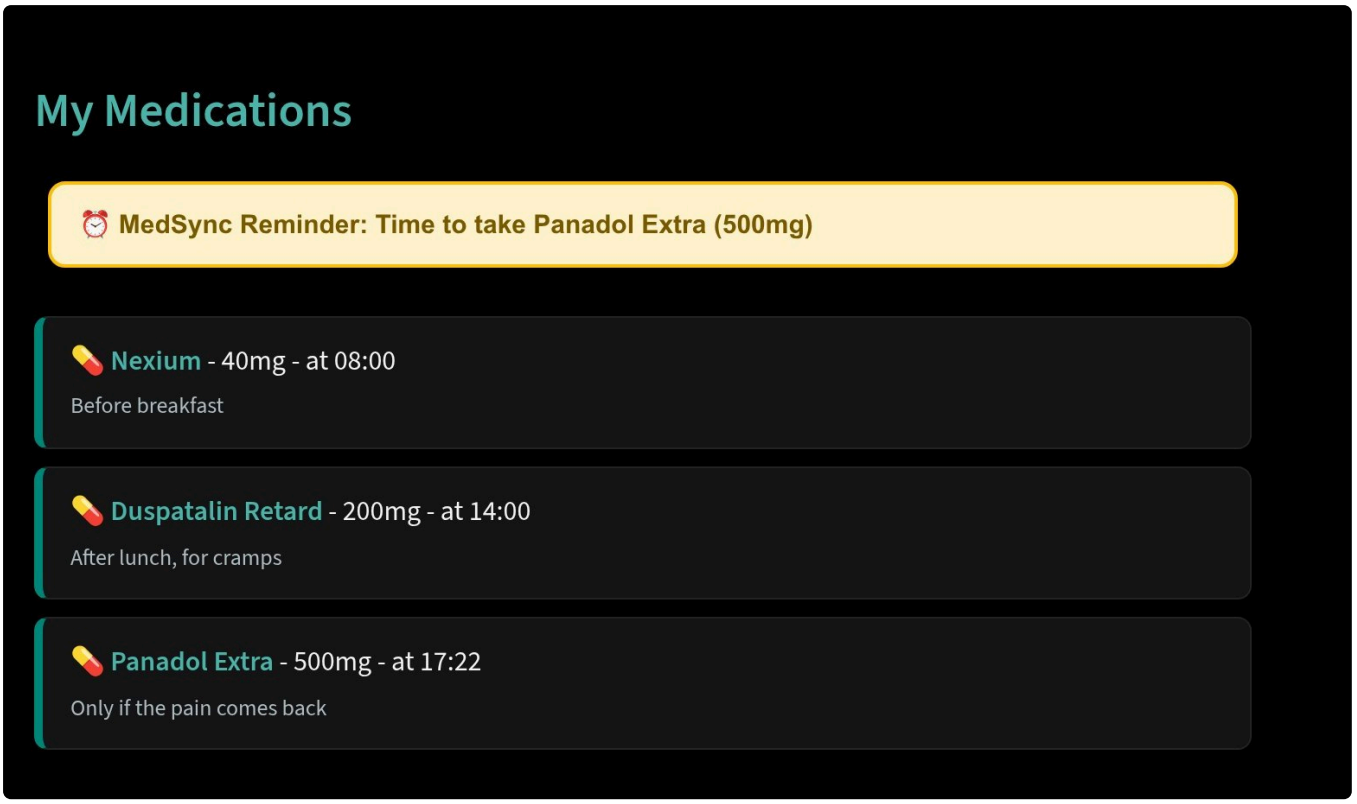
What data is stored?

9 tables: accounts and the extra details for each role, care teams, medicines, prescriptions, the dose log and visit notes.

What We Learned | ماذا تعلمنا

1. **The hardest part:** repairing the database. The original `database.db` had patient rows pointing to accounts that didn't exist, and tables were missing, so we wrote `repair_orphan_records()` and added the new tables.
2. **The most fun part:** the chatbot, and seeing Gemini answer health questions.
3. **If we had more time:** a phone alarm that rings even when the app is closed.
4. **One thing we would do differently:** design the database first, with real foreign keys, before writing the pages.

Screenshot of Our Project | لقطة من مشروعنا



Sief & Leen · AFFIAT · Afro-Asian Forum for Innovation and Technology · Aswan Finals 2026